

软件需求管理

文档类型： C语言编程规范
适用范围： 公司内使用C语言编码的所有软件
生效日期： 自发布之日起生效
版本信息： V1.0

文档变更记录

版本号	变更日期	变更人	评审人	变更描述
V1.0	2025-11-20	王天楠	杜龙彬	初始版本发布，包含基础

文档概述

本规范制定了编写C语言程序的基本原则、规则和建议。从代码的清晰、简洁、可测试、安全、程序效率、可移植各个方面对C语言编程作出了具体指导。

规范组成

- **原则：** 编程时必须坚持的指导思想
- **规则：** 编程时强制必须遵守的约定
- **建议：** 编程时必须加以考虑的约定

主要内容

1. 头文件规范
2. 函数设计规范
3. 标识命名与定义
 - 变量命名
 - 宏命名
 - 函数命名
 - 结构体命名

4. 排版与格式
5. 注释规范
6. 可测性要求
7. 中断处理规范

实施要求

- 新编写代码必须遵守本规范
- 修改现有代码时应逐步符合本规范
- 所有开发人员需熟悉并执行本规范

本文档版权归XX有限公司所有，未经许可不得外传

编制部门：软件开发部

编制日期：2025年11月

密级：内部公开

C语言编程规范

范围

本规范适用于公司内使用C语言编码的所有软件。本规范自发布之日起生效，以后新编写的和修改的代码应遵守本规范。

简介

本规范制定了编写C语言程序的基本原则、规则和建议。从代码的清晰、简洁、可测试、安全、程序效率、可移植各个方面对C语言编程作出了具体指导。

0 规范制订说明

0.1 前言

为提高产品代码质量，指导广大软件开发人员编写出简洁、可维护、可靠、可测试、高效、可移植的代码，编程规范修订工作组分析、总结了我司的各种典型编码问题，并参考了业界编程规范近年来的成果，重新对我司编程规范进行了梳理、优化、刷新，编写了本规范。本规范将分为完整版和精简版，完整版将包括更多的样例、规范的解释。

0.2 代码总体原则

1. 清晰第一

清晰性是易于维护、易于重构的程序必需具备的特征。代码首先是给人读的，好的代码应当可以像文章一样发声朗诵出来。

“程序必须为阅读它的人而编写，只是顺便用于机器执行。”

——Harold Abelson 和 Gerald Jay Sussman

“编写程序应该以人为本，计算机第二。”

——Steve McConnell

2. 简洁为美

简洁就是易于理解并且易于实现。代码越长越难以看懂，也就越容易在修改时引入错误。

3. 选择合适的风格，与代码原有风格保持一致

产品所有人共同分享同一种风格所带来的好处，远远超出为了统一而付出的代价。

0.3 规范实施、解释

本规范适用于公司内使用C语言编码的所有软件。本规范自发布之日起生效，对以后新编写的和修改的代码应遵守本规范。

0.4 术语定义

- **原则**：编程时必须坚持的指导思想
- **规则**：编程时强制必须遵守的约定
- **建议**：编程时必须加以考虑的约定
- **说明**：对此原则/规则/建议进行必要的解释
- **示例**：对此原则/规则/建议从正、反两个方面给出例子

1 头文件

头文件的设计体现了大部分的系统设计。不合理的头文件布局是编译时间过长的根因。

定义

- **依赖**：若x.h包含了y.h，则称作x依赖y。依赖关系会进行传导，如x.h包含y.h,而y.h又包含了z.h，则x通过y依赖了z。依赖将导致编译时间的上升。虽然依赖是不可避免的，也是必须的，但是不良的设计会导致整个系统的依赖关系无比复杂，使得任意一个文件的修改都要重新编译整个系统，导致编译时间巨幅上升。

原则

1.1 头文件中适合放置接口的声明，不适合放置实现

说明：

- 头文件是模块或单元的对外接口，应放置对外部的声明。
- 变量定义不应放在头文件中，应放在.c文件中。变量的声明尽量不要放在头文件中，亦即尽量不要使用全局变量作为接口。变量是模块或单元的内部实现细节,不应通过在头文件中声明的方式直接暴露给外部,应通过函数接口的方式进行对外暴露。即使必须使用全局变量，也只应当在.c中定义全局变量，在.h中仅声明变量为全局的

1.2 头文件应当职责单一

说明：头文件过于复杂，依赖过于复杂是导致编译时间过长的主要原因。很多现有代码中头文件过大，职责过多，再加上循环依赖的问题，可能导致为了在.c中使用一个宏，而包含十几个头文件。

1.3 头文件应向稳定的方向包含

说明：头文件的包含关系是一种依赖，一般来说，应当让不稳定的模块依赖稳定的模块，从而当不稳定的模块发生变化时，不会影响(编译)稳定的模块。

规则

1.1 每一个 .c 文件应有一个同名 .h 文件

说明：用于 .h 声明需要对外公开的接口。

1.2 禁止头文件循环依赖

说明：头文件循环依赖，指a.h包含b.h，b.h包含c.h，c.h包含a.h之类导致任何一个头文件修改，都导致所有包含了a.h/b.h/c.h的代码全部重新编译一遍。而如果是单向依赖，如a.h包含b.h，b.h包含c.h，而c.h不包含任何头文件，则修改a.h不会导致包含了b.h/c.h的源代码重新编译

1.4 头文件应当自包含

说明：任意一个头文件均可独立编译。

1.5 总是编写内部 `#include` 保护符（`#define` 保护）

说明：使用 `#define` 防止头文件被多重包含，命名格式为 `FILENAME_H`，建议使用 `PROJECTNAME_PATH_FILENAME_H`。

1.6 禁止在头文件中定义变量

1.7 只能通过包含头文件的方式使用其他 `.c` 提供的接口

说明：禁止在 `.c` 中通过 `extern` 的方式使用外部函数接口、变量。

2 函数

原则

2.1 一个函数仅完成一件功能

说明：一个函数实现多个功能给开发、使用、维护都带来很大的困难。将没有关联或者关联很弱的语句放到同一函数中，会导致函数职责不明确，难以理解，难以测试和改动。

2.2 重复代码应该尽可能提炼成函数

说明：复代码提炼成函数可以带来维护成本的降低。

规则

2.1 避免函数过长，新增函数不超过50行（非空非注行）

例外：某些实现算法的函数可能会超过50行。

2.2 避免函数的代码块嵌套过深，新增函数的代码块嵌套不超过4层

2.3 可重入函数应避免使用共享变量；若需使用，应通过互斥手段保护

2.4 废弃代码（没有被调用的函数和变量）要及时清除

说明：程序中的废弃代码不仅占用额外的空间，而且还常常影响程序的功能与性能，很可能给程序的测试、维护等造成不必要的麻烦。

2.5 在源文件范围内声明和定义的所有函数，除非外部可见，否则应增加 `static` 关键字

说明：如果一个函数只是在同一文件中的其他地方调用，那么就用static声明。使用static确保只是在声明它的文件中是可见的，并且避免了和其他文件或库中的相同标识符发生混淆的可能性。

2.6 函数的参数个数不超过5个。

说明：函数的参数过多，会使得该函数易于受外部(其他部分的代码)变化的影响，从而影响维护工作。函数的参数过多同时也会增大测试的工作量。函数的参数个数不要超过5个，如果超过了建议拆分为不同函数。

3 标识命名与定义

原则

3.1 标识符的命名要清晰、明了，有明确含义，同时使用完整的单词或大家基本可以理解的缩写，避免使人产生误解。

说明：尽可能给出描述性名称，不要节约空间，让别人很快理解你的代码更重要。

示例：

```
// 好的命名
int error_number;
int number_of_completed_connection;

// 不好的命名
int n;
int nerr;
int n_comp_conns;
```

3.2 用正确的反义词组命名具有互斥意义的变量或相反动作的函数等。示例:

```
add/remove    up/down          create/destroy
min/max       next/previous    send/receive
```

3.3 尽量避免名字中出现数字编号，除非逻辑上的确需要编号示例:如下命名，使人产生疑惑。

```
#define EXAMPLE _0_ TEST
#define EXAMPLE _1_ TEST
```

应改为有意义的单词命名

```
#define EXAMPLE_UNIT_TEST
#define EXAMPLE_ASSERT_TEST
```

- 3.4 命名风格要自始至终保持一致。
- 3.5 命名中若使用特殊约定或缩写，要有注释说明。
- 3.6 为了代码复用，命名中应避免适用与具体项目相关的前缀。

4 文件命名规则

原则

- 4.1 文件与目录规范：结构清晰，避免冲突。
- 4.2 文件和目录命名需体现模块功能，减少跨模块依赖，便于团队协作维护。

规则

- 4.1 源文件 (.c)、头文件 (.h) 均用小写，单词间用_分隔；目录按功能划分（如 drivers驱动层、system系统层）。

```
project/  
├─ drivers/           // 硬件驱动模块（UART、SPI、GPIO等）  
│   ├─ uart_driver.c  
│   └─ spi_driver.h  
├─ system/           // 系统核心模块（任务调度、看门狗等）  
│   ├─ task_scheduler.c  
│   └─ watchdog.h  
├─ application/      // 业务应用模块（传感器处理、显示控制等）  
│   └─ sensor_process.c  
└─ main.c            // 主函数（模块调度入口）
```


5 变量定义

原则

5.1 变量命名：小驼峰式（区分作用域）首字母小写，后续单词首字母大写；通过前缀 / 修饰符区分作用域（局部 / 静态局部）。

变量命名规则： <范围>_<类型>名称

范围前缀表

范围	前缀	示例
结构体成员变量	m_	m_redComponent
静态变量	s_	s_instanceCount
全局变量	g_	g_systemTick
局部变量	无前缀	tempValue

类型前缀表

类型	前缀	示例
指针	p	pBuffer
枚举	e	eDeviceState
结构体	t	tSensorData
布尔	b	bIsReady
浮点	f	fTemperature
双精度	d	dPrecisionValue
字符	c	cKeyInput
无符号整型	u	uDataLength
有符号整型	i	iCounter
短整型	s	sSmallValue

类型	前缀	示例
长整型	l	lLongValue

5.2局部循环体控制变量优先使用i、j、k等；局部长度变量优先使用len、num等；临时中间变量优先使用temp、tmp等

示例：

```
// 单层循环
for (uint8_t i = 0; i < 10; i++)
{
    printf("%d ", i);
}
```

正确示例

```
// 正确：局部变量，表示传感器数值
uint32_t sensorValue;

// 正确：静态局部变量，表示数据包计数器
static uint8_t packetCounter;
```

错误示例

```
// 错误：蛇形命名+大驼峰混用
int Sensor_Value;

// 错误：局部变量使用大驼峰
float Temperature;
```

5.3 全局变量：带g_前缀。

说明：所有全局变量（含静态全局变量）加g_前缀，避免跨文件命名冲突；硬件相关变量需用volatile修饰，防止编译器优化。

正确示例

```
// 正确：全局变量，系统滴答计数器（volatile修饰硬件变量）  
volatile uint32_t g_systemTickCount;
```

```
// 正确：静态全局变量，数据包错误计数器  
static uint16_t g_packetErrorCount;
```

错误示例

```
// 错误：缺少g_前缀，在变量较多的情况下较难识别为全局变量  
  
uint32_t SystemTick;
```

6 宏的命名规则

原则

6.1 对于数值或者字符串等等常量的定义，建议采用全大写字母，单词之间加下划线”的方式命名(枚同样建议使用此方式定义)。

示例：

```
#define PI_ROUNDED 3.14
```

7 函数命名：大驼峰式（动词开头）

7.1 首字母大写，后续单词首字母大写；以动词开头（如Init、Read、Configure），明确函数动作。

正确示例

```
// 正确：明确初始化UART外设，符合大驼峰+动词开头
void InitUartPeripheral(void);
```

错误示例

```
// 错误：蛇形命名不符合规范
void init_uart(void);
// 错误："temperature"首字母未大写，大小写不一致
void ReadtemperatureSensor(void);
```

8 自定义类型命名规范

规定

- 结构体类型：使用 t 前缀 + 大驼峰命名法，明确标识结构体类型
- 枚举类型：使用 e 前缀 + 大驼峰命名法，明确标识枚举类型
- 类型命名：首字母前缀清晰标识类型种类，便于代码阅读和维护

示例：

```
// 类型定义
typedef enum {
    eSENSOR_OFF,
    eSENSOR_STANDBY,
    eSENSOR_ACTIVE
} eSensorState;

typedef struct {
    int nId;
    char szName[50];
    float fCurrentValue;
    eSensorState eState;
} tSensor;

// 全局传感器列表
tSensor g_arrSensors[MAX_SENSORS];
```

9 排版与格式

规则

9.1程序块采用缩进风格编写，每级缩进为4个空格。

说明：当前各种编辑器/IDE都支持TAB键自动转空格输入，需要打开相关功能并设置相关功能。编辑器，如果有显示TAB的功能也应该打开，方便及时纠正输入错误。

规则

9.2相对独立的程序块之间、变量说明之后必须加空行。

规则

9.3一条语句不能过长，如不能拆分需要分行写。一行到底多少字符换行比较合适，产品可以自行确定。

换行时有如下建议：

- 换行时要增加一级缩进，使代码可读性更好
- 低优先级操作符处划分新行
- 换行时操作符应该也放下来，放在新行首
- 换行时建议一个完整的语句放在一行，不要根据字符数断行

示例：

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;

    int result = a + b + 100 + 200 + 300 + 400 +
                500 + 600 + 700 + 800 + 900 + 1000;

    printf("The result is %d", result);

    return 0;
}

// 长表达式在低优先级操作符处换行
int complex_result = (a * b) + (c * 2) +
                    (a + b + c) * 3 -
                    (a - b) / 2;
```

10 中断处理函数规范

原则

中断处理函数应遵循**极简主义**和**规范命名**的原则，确保中断响应及时、可靠。

规则

函数名必须严格遵循芯片厂商规范，确保中断向量正确映射。

示例：

- USART1_IRQHandler - USART1 中断服务函数
- TIM2_IRQHandler - 定时器2 中断服务函数
- EXTI0_IRQHandler - 外部中断0 服务函数

代码规范

规则1：函数命名标准化

```
// 正确：遵循芯片厂商命名规范
void USART1_IRQHandler(void);
void TIM2_IRQHandler(void);
void EXTI0_IRQHandler(void);
```

```
// 错误：自定义命名会导致中断无法响应
void MyUartInterrupt(void);
void Timer_ISR(void);
```

11 注释规范

原则

说明： 有助于对程序的阅读理解，说明程序在"做什么"，解释代码的目的、功能和采用的方法。一般情况源程序有效注释量在30%左右。 注释语言必须准确、易懂、简洁。边写代码边注释，修改代码同时修改相应的注释，不再有用的注释要删除。 汇编和C中都用"//"，取消";" 不使用段注释"/* */"（调试时可用）

11.1 文件头注释：必含核心信息

说明： 每个.c和.h文件开头加文件头注释，包含文件名、项目名称、版权、功能、作者、日期等信息。

```
/*
*****
 * @copyright: © 2025, XX有限公司. All rights reserved.
 * @file: websocket.c
 * @description: 实现485协议的数据解析、帧封装与发送逻辑
 * @author: XXX
 * @date: 2025-08-14
 * @version: V1.0
*****
*/
```

11.2 函数注释：复杂函数必加

说明： 复杂函数（如算法、硬件配置）需加注释，说明核心逻辑；简单函数（如变量赋值）可省略。

```
/**
 * @brief 计算CRC16-CCITT校验值（适用于串口数据校验）
 * @note 多项式: 0x1021 (x^16 + x^12 + x^5 + 1)，初始值: 0xFFFF，无反码
 * @param pData: 待校验数据指针（输入，非NULL）
 * @param length: 待校验数据长度（输入，>0）
 * @return uint16_t: 计算后的CRC值
 */
uint16_t CalculateCrc(const uint8_t* pData, uint16_t length) {
    // ... 逻辑实现
}
```


11.3代码注释

注释规范

规则

代码注释应与被注释的代码紧邻，放在其上方或右方，不可放在下面。

说明

- 如放于上方则需与其上面的代码用空行隔开
- 一般少量注释应该添加在被注释语句的行尾
- 一个函数内的多个注释左对齐
- 较多注释则应加在上方且注释行与被注释的语句左对齐

注释位置示例

上方注释（较多内容时）

```
// 空行隔开
/**
 * @brief 初始化SPI外设
 * @note 配置为主机模式，时钟极性为0，相位为0
 */
void InitSPI(void)
{
    SPI_InitTypeDef spi_init;
    spi_init.Mode = SPI_MODE_MASTER;
    spi_init.CLKPolarity = SPI_POLARITY_LOW;
    spi_init.CLKPhase = SPI_PHASE_1EDGE;
    // ... 其他配置
}
```

多行注释左对齐

```
void HandleNetworkPacket(void)
{
    ParseHeader(packet);      // 解析数据包头部
    ValidateChecksum(packet); // 验证校验和
    ProcessPayload(packet);   // 处理有效载荷
    SendAck(sender_id);       // 发送确认信号
}
```

分支语句注释规则

```
// 遍历传感器数组，读取并处理数据
for (int i = 0; i < SENSOR_COUNT; i++)
{
    // 检查传感器是否在线
    if (sensors[i].status == ONLINE)
    {
        // 读取传感器原始数据
        raw_data = ReadSensor(i);

        // 应用数据滤波算法
        filtered_data = ApplyFilter(raw_data);

        // 更新显示数据
        UpdateDisplay(i, filtered_data);
    }
}
```

软件可测性设计原则

原则12.1：模块可测性设计

核心要求：模块划分清晰，接口明确，耦合性小，有明确输入和输出，否则单元测试实施困难。

单元测试实施的依赖条件

1. 接口定义规范

- **定义清楚**：接口参数、返回值类型明确
- **定义完整**：涵盖所有必要的输入输出
- **定义稳定**：接口变更频率低，向后兼容

2. 功能验收条件明确

- **预置条件**：执行前的环境状态要求
- **输入数据**：明确的输入参数范围和格式
- **预期结果**：可验证的输出结果标准

3. 内部状态可观测

- **状态可查询**：模块内部关键状态可获取
- **数据可修改**：测试时能够设置关键数据
- **过程可监控**：执行过程可追踪

4. 执行路径控制

- **入口唯一**：原子功能只有一个明确入口点
- **出口唯一**：原子功能有确定的出口点
- **路径清晰**：执行流程可预测

5. 依赖管理

- **依赖集中**：外部依赖集中处理
- **全局变量最小化**：
 - 减少全局变量使用
 - 或采用封装形式（如单例、服务定位器）
 - 提供可控的依赖注入机制